TON Simplex Consensus — Security Findings

Repository: ton-blockchain/ton **Branch:** testnet **Commit:** `3bb6abc` **Scope:** validator/consensus/simplex/ (primary) · overlay/ (secondary) **Date:** 2026-03-14

Findings: 5 total — 2 HIGH (primary) · 2 MEDIUM (primary) · 1 MEDIUM (secondary) **Dropped:** 2 false positives confirmed via code analysis prior to submission

---

HIGH Finding 1 — Honest Validator Self-Equivocation: SkipVote + NotarizeVote for the Same Slot

Scope: Primary — `validator/consensus/simplex/`

Summary

Three linked bugs in `consensus.cpp` and `pool.cpp` allow an honest validator to broadcast both a `SkipVote` and a `NotarizeVote` for the same slot during normal operation. This equivocation is also silently ignored by peers — no `MisbehaviorReport` is generated. Under conditions where block validation is slower than the alarm interval (high network load, heavy blocks, slow hardware), this affects every validator simultaneously, causing complete liveness failure for affected consensus rounds.

Root Cause

`try_notarize()` is an async coroutine that suspends at three `co_await` points before setting `voted_notar` and broadcasting `NotarizeVote`. The `alarm()` timer fires independently, with an insufficient guard:

```
// consensus.cpp — alarm()
if (slot && !slot->state->voted_final) {     // BUG A: should also check
!voted_notar && !voted_skip
    owning_bus().publish<BroadcastVote>(SkipVote{i}).start().detach();
    slot->state->voted_skip = true;
}
```

Meanwhile `try_notarize()` does not check `voted_skip` before issuing its vote after resuming from validation:

```
// consensus.cpp — try_notarize() (after all co_await points)
slot.state->voted_notar = candidate->id;     // set after awaits complete
owning_bus().publish<BroadcastVote>(NotarizeVote{candidate->id})...;
// BUG B: no check for voted_skip before this point
```

Race: alarm fires during the `ValidationRequest` suspension → `SkipVote` broadcast, `voted_skip = true` → coroutine resumes → `NotarizeVote` broadcast.

The same honest validator has now broadcast conflicting votes.

The third bug (**Bug C**) prevents any peer from detecting this:

```
// pool.cpp — Tsentrizbirkom::check_invariants()
if (notarize_.has_value() && finalize_.has_value() && notarize_->vote.id !=
finalize_->vote.id)
    return ConflictingVotes::create(...);
if (finalize_.has_value() && skip_.has_value())
    return ConflictingVotes::create(...);
// MISSING: NotarizeVote + SkipVote is also conflicting in Simplex
// if (notarize_.has_value() && skip_.has_value()) → ConflictingVotes
```

Demonstrated Impact

Tested on commit `3bb6abc` with 8 equal-weight validators, 1.5–2.5s validation time, 200ms block target. All 8 validators equivocated on every slot where validation exceeded the alarm window:

PoC Output (pool.cpp:525 — WARNING):

`Dropping NotarizeVote{id={60, +7z0gpDDmAsyzTc3oLsh7f+82mx3OuXOMCuNHuCwvOM=}} from validator 2 at lzPRai9kr3RHHmjlDhvucvghdTtfT0W1EsHWV7G+Muk= which references a finalized slot`

Slot 60 was already finalized via `SkipCert` when this `NotarizeVote` arrived, proving validator 2 broadcast both `SkipVote{60}` and `NotarizeVote{60}`. Same pattern observed across all 8 validators.

Liveness consequence: Equivocating validators have `voted_skip = true`, which blocks `try_vote_final()` from issuing `FinalizeVote`. Slots that would have been finalized via `FinalCert` are instead skipped, losing the candidate block. In a 30-second run, the equivocation caused severe throughput degradation — all candidate blocks for affected slots were abandoned.

Additionally, a validator that has equivocated contributes its weight to both `notarize_weight` and `skip_weight` for the same slot, distorting the weight accounting used to form certificates.

Reproduction

```
git clone --branch testnet https://github.com/ton-blockchain/ton.git
cd ton && mkdir build-clang && cd build-clang
CC=clang CXX=clang++ cmake .. -DCMAKE_BUILD_TYPE=Release -
DTON_USE_ROCKSDB=OFF
cmake --build . --target test-consensus -j$(nproc)

# Requires Clang 18+ — GCC cannot compile this codebase
./test/consensus/test-consensus \
  --validation-time 1.5:2.5 \
  --target-rate-ms 200 \
  --n-nodes 8 \
```

```
  --duration 30 \
  -v 2 2>&1 | grep "Dropping NotarizeVote.*finalized slot"
```

Expected: multiple `WARNING [pool.cpp:525]` lines showing `NotarizeVote` arriving for already-skipped slots, confirming each validator sent both vote types.

Fix

Bug A — add `voted_notar` and `voted_skip` guards in `alarm()`:

```
if (slot && !slot->state->voted_final
         && !slot->state->voted_notar    // new
         && !slot->state->voted_skip) {  // new
    owning_bus().publish<BroadcastVote>(SkipVote{i})...;
    slot->state->voted_skip = true;
}
```

Bug B — check `voted_skip` in `try_notarize()` after all suspension points:

```
if (slot.state->voted_skip) { co_return {}; }  // new guard before voting
slot.state->voted_notar = candidate->id;
owning_bus().publish<BroadcastVote>(NotarizeVote{candidate->id})...;
```

Bug C — add the missing case to `check_invariants()`:

```
if (notarize_.has_value() && skip_.has_value()) {
    return ConflictingVotes::create(
        notarize_->serialize_as_proof(), skip_->serialize_as_proof());
}
```

HIGH

Finding 2 — start_up() Bootstrap Replay Equivocates on Every Validator Restart

Scope: Primary — `validator/consensus/simplex/consensus.cpp`

Summary

The `ConsensusImpl` constructor applies the same missing guard as the `alarm()` race (Finding 1) but fires **deterministically on every validator restart**, not under a race condition. Any validator that crashes or is restarted after casting a `NotarizeVote` in an unfinished leader window immediately broadcasts a `SkipVote` for the same slot on startup.

Root Cause

During startup, previously cast votes are loaded from persistent storage into in-memory state (lines 47–80). Then the current unannouncedleader window broadcasts `SkipVote` for all non-final slots (lines 82–91):

```
// consensus.cpp — start_up() bootstrap phase, lines 54–62: voted_notar
restored
auto notar_fn = [&](const NotarizeVote& notar_vote) {
  slot->state->voted_notar = notar_vote.id;   // loaded from DB
};
...
std::visit(td::overloaded(notar_fn, final_fn, skip_fn), vote.vote);

// consensus.cpp — start_up() window broadcast, lines 82–91
if (auto window = bus.first_nonannounced_window) {
  for (td::uint32 i = start_slot; i < end_slot; ++i) {
    auto slot = state_->slot_at(i);
    if (slot.has_value() && !slot->state->voted_final) {  // BUG: must
check !voted_notar
      slot->state->voted_skip = true;
      owning_bus().publish<BroadcastVote>(SkipVote{i}).start().detach();
    }
  }
}
```

If `voted_notar` was loaded for slot X from the database, and slot X is in the current unannouncedwindow, the guard `!voted_final` passes (the slot was never finalised — that's why the window is unannouncedand this is a fresh start) and `SkipVote{X}` is broadcast. The pre-crash `NotarizeVote{X}` remains in peers' pools. Equivocation is the result.

Trigger Scenario

This is the third instance of the same missing-guard bug, but it fires on every restart where the following state exists in the database:

1. Validator was the leader for window W, sent `NotarizeVote{slot X}`, and persisted it.
2. Window W had not yet advanced (`first_nonannounced_window = W` still in DB).

3. Validator is restarted for any reason (OOM, crash, planned maintenance).

Unlike the runtime alarm() race, this requires no timing coincidence — it fires unconditionally.

Fix

```
if (slot.has_value()
    && !slot->state->voted_final
    && !slot->state->voted_notar) {    // new guard — same fix as alarm()
  slot->state->voted_skip = true;
  owning_bus().publish<BroadcastVote>(SkipVote{i}).start().detach();
}
```

MEDIUM

Finding 3 — ConflictingCandidateAndCertificate Stores No Evidence

Scope: Primary — `validator/consensus/simplex/pool.cpp`, `misbehavior.h`

Multiple call sites in `pool.cpp` create `ConflictingCandidateAndCertificate` misbehavior reports via a factory that takes no arguments and stores no cryptographic evidence:

```
// misbehavior.h
class ConflictingCandidateAndCertificate : public Misbehavior {
 public:
  static MisbehaviorRef create() { // no candidate or certificate stored
    return td::make_ref<ConflictingCandidateAndCertificate>();
  }
};
```

There are four call sites in `pool.cpp` (lines 646, 656, 666, 676), all with the evidence arguments commented out:

```
return resolve_with(ConflictingCandidateAndCertificate::create(
    /* candidate, last_finalization_cert */));   // evidence not passed
```

A `MisbehaviorReport` without cryptographic evidence cannot support on-chain slashing. The misbehavior is detected locally but cannot be proven to other nodes or the elector contract. A Byzantine leader can propose candidates conflicting with existing certificates with no deterrent effect.

MEDIUM

Finding 4 — Three Misbehavior Detection FIXMEs Never Implemented

Scope: Primary — `validator/consensus/simplex/consensus.cpp`

Three detectable misbehavior conditions in `consensus.cpp` are silently dropped with `// FIXME: Report misbehavior` comments:

Line	Condition	Protocol Violation
174	Candidate with `parent_slot ≥ own_slot`	Invalid block structure
180	Leader proposes two different candidates for the same slot	Proposal equivocation
227	Candidate fails full block validation (`CandidateReject`)	Invalid block proposal

Byzantine leaders can exploit all three silently dropped cases to degrade liveness — forcing re-validation, occupying `ValidationRequest` slots, wasting bandwidth — without any on-chain consequence.

MEDIUM | Secondary Scope

Finding 5 — TwoStep Broadcast: Deduplication Check Placed After Rebroadcast

Scope: Secondary — `overlay/broadcast-twostep.cpp`

Summary

In both the simple and FEC broadcast paths, `rebroadcast()` is called before the `is_delivered()` deduplication check. Any node can replay a previously seen broadcast within the ±20-second timestamp window and cause every relay node to forward the duplicate to all N−1 peers before recognising it as a duplicate.

Vulnerable Code

```
// broadcast-twostep.cpp — simple path, lines 313–316
if (will_rebroadcast) {
    rebroadcast(overlay, bcast_src_adnl_id, serialize_tl_object(broadcast,
true));  // line 314 — relays to N-1 peers
}
if (overlay->is_delivered(broadcast_id)) { return; }  // line 316 — dedup
check too late

// broadcast-twostep.cpp — FEC path, lines 346–352 (same pattern)
if (will_rebroadcast) {
    rebroadcast(overlay, bcast_src_adnl_id, serialize_tl_object(broadcast,
true));  // line 347
}
if (overlay->is_delivered(broadcast_id)) { return; }  // line 352 — dedup
check too late
```

`rebroadcast()` iterates all peers with no per-sender deduplication or rate limiting. The only guard is a ±20-second timestamp check, giving a 40-second replay window per broadcast.

Impact

Per replay: (N−1) × (N−2) messages across the overlay. For N=300: 89,402 messages per duplicate. No authentication or uniqueness check prevents a Byzantine node from replaying within the window.

Fix

```
if (overlay->is_delivered(broadcast_id)) { return; }     // check first
overlay->register_delivered_broadcast(broadcast_id);
if (will_rebroadcast) {
    rebroadcast(overlay, bcast_src_adnl_id, ...);         // then relay
}
```

Build Notes

- GCC 13 cannot compile the codebase — template specialisations and coroutine syntax require **Clang 18+**
- Applied upstream-compatible compilation fixes: `co_return {} → co_return td::Unit{}` in three archiver files
- All findings tested on commit `3bb6abc`, branch `testnet`

Submitted for Blockchain Contest Round 2 — TON Simplex Consensus security review. Contact: TesseraeVentures.